Assignment Beta — Question 2 Report

Parameter-Efficient Fine-Tuning for E-commerce Function Calling

Detail	Value
Student ID	23K0069
Course	AI4015 — Agentic AI, Fall 2026
Base model	Qwen/Qwen2.5-1.5B-Instruct
Methods compared	Base model, LoRA, QLoRA, DoRA, AdaLoRA
Hardware	NVIDIA Tesla T4, 14.56 GB VRAM
Training precision	FP16
Random seed	42

Executive summary

The untouched Qwen2.5-1.5B-Instruct model achieved the strongest exact function-call match on the notebook's 200-example test set: **41.5% (83/200)**. Fine-tuning improved function-name selection to 98–100%, but exact function-call accuracy fell to 24–25%. This difference matters: a function call is useful only when the selected tool and its arguments are both correct.

QLoRA was the most memory-efficient training method, using 1.56 GB peak allocated VRAM, but it did not improve exact-match accuracy. Plain LoRA trained fastest among the adapters. DoRA took the longest. The rank sweep from r=4 to r=32 did not improve exact-match accuracy.

Recommendation: none of the evaluated models is reliable enough for autonomous transaction execution. For a limited, human-supervised pilot, the base model is the least-bad candidate because it achieved the highest exact-match score and schema validity. Every proposed action should still pass deterministic backend validation, and refunds or ambiguous cases should require human approval.

1. Experimental setup and data limitation

The experiment used Qwen2.5-1.5B-Instruct, the same random seed, training settings and held-out test set across the baseline and four adaptation methods. The notebook attempted to curate 2,000 compatible examples from Glaive Function Calling V2. Only **two** examples could be mapped reliably to the assignment's four e-commerce tools. The notebook therefore activated its separately labelled **synthetic fallback** and generated 2,000 template-based examples: 1,800 training examples and 200 held-out test examples, balanced across the four tools. The split used disjoint prompt-template families, with no identical customer-message overlap between training and test.

This means the measurements below are a controlled comparison on synthetic examples, **not** evidence of performance on Glaive examples or real customer-support conversations. That is the most important limitation when interpreting the results.

Training used three epochs, FP16 on a Tesla T4, a learning rate of 2e-4, per-device batch size 1 and gradient accumulation of 8. The base model was evaluated without fine-tuning. LoRA, QLoRA and DoRA used Unsloth; AdaLoRA used the native Hugging Face PEFT adaptive-rank implementation.

2. Results

2.1 Quality on the same 200 held-out examples

Method	Valid JSON	Correct tool	Valid argument schema	Exact full match
Base model	100%	87.5% (175/200)	99.0% (198/200)	**41.5% (83/200)**
LoRA, r=8	100%	**100% (200/200)**	25.0% (50/200)	25.0% (50/200)
QLoRA, r=8	100%	99.5% (199/200)	25.0% (50/200)	25.0% (50/200)
DoRA, r=8	100%	98.0% (196/200)	25.0% (50/200)	25.0% (50/200)
AdaLoRA, initial r=12, target r=8	100%	99.0% (198/200)	24.5% (49/200)	24.0% (48/200)

JSON validity only measures whether the output parses as one JSON object. Tool accuracy measures whether the function name is correct. Schema validity checks whether the selected tool has exactly its allowed argument keys and non-empty string values. Full match requires both the correct tool and exact equality of every argument with the held-out reference.

The distinction explains the apparently contradictory results: adapters nearly always selected the right function, but only about one quarter of their outputs passed the argument-schema check. A syntactically valid JSON response is therefore not equivalent to a safe, executable function call.

2.2 Training cost and GPU memory

Method	Trainable parameters	Peak allocated VRAM	Training time
Base model	0	2.91 GB peak evaluation VRAM	Not applicable
LoRA	2,179,072	2.99 GB	17.83 min
QLoRA	2,179,072	**1.56 GB**	22.66 min
DoRA	2,279,424	3.00 GB	56.40 min
AdaLoRA	3,270,064	3.04 GB	26.99 min

QLoRA reduced measured peak allocated training memory by roughly 48% relative to LoRA (1.56 GB versus 2.99 GB), but took about 4.8 minutes longer and did not improve exact-match accuracy. DoRA required the longest training time in this run. The base model's 2.91 GB figure

is evaluation memory, not training memory, so it should not be compared as though a baseline training run occurred.

The notebook did **not** record tokens per second, time to first token or end-to-end inference latency. Therefore, no measured inference-speed winner can be claimed. Before deployment, the exact serving artifact must be benchmarked under the intended batch size, context length and adapter configuration.

3. Rank experiment: does a larger LoRA rank help?

LoRA was selected as the best PEFT method for the rank sweep because it had the highest PEFT exact-match result (tied at 25%) and the shortest adapter training time.

LoRA rank	Tool accuracy	Valid argument schema	Exact full match	Peak allocated VRAM	Training time
4	99.0%	25.0%	25.0%	2.97 GB	18.14 min
8	100%	25.0%	25.0%	2.99 GB	17.83 min
16	97.5%	25.0%	25.0%	3.02 GB	17.93 min
32	100%	25.0%	25.0%	3.08 GB	17.97 min

Increasing rank from 4 to 32 increased the trainable-parameter share of the base-model reference from approximately 0.071% to 0.565%, and peak allocated VRAM rose slightly. However, exact-match accuracy remained fixed at 25% and schema validity remained fixed at 25%. The experiment therefore provides no evidence that a larger rank solved the observed argument-generation problem. Rank 8 is the more economical choice for this particular run; increasing rank would add parameters without a measured gain in the metric that matters most.

4. Production decision

If one candidate had to be used next Monday, I would choose the **untuned base model for a restricted, human-supervised pilot**, not for autonomous execution. It had the highest exact-match score (41.5%), the highest schema-validity score (99%) and the lowest measured evaluation memory (2.91 GB). It also avoids the adapter-training step. However, 83 correct full calls out of 200 is not an acceptable autonomous transaction success rate. The model must not be allowed to execute refunds or other consequential actions without validation and appropriate approval.

Among the PEFT options, plain **LoRA** is the most practical starting point for another experiment: it matched the best adapter exact-match score (25%) and had the shortest measured training time (17.83 minutes). QLoRA is preferable only when training memory is the binding constraint; its lower VRAM did not translate into better task accuracy. DoRA's 56.40-minute training time was difficult to justify in this experiment, and AdaLoRA did not outperform the fixed-rank alternatives.

The merging detail also matters. QLoRA trains adapters over a quantized base. When an adapter is merged into quantized weights, the documented PEFT path dequantizes the base weights, adds the adapter update and quantizes the result again; that re-quantization can introduce rounding error, and repeated merge/unmerge cycles can accumulate it. Keep the original quantized checkpoint and test the merged artifact separately. DoRA decomposes each

weight update into magnitude and direction: the direction is handled with a LoRA-style update and magnitude is learned separately. Its merge must incorporate both parts into the dense base weights. PEFT recommends merging DoRA for inference where supported because the unmerged method has additional runtime overhead. These are deployment considerations from the library documentation; the notebook did not run merged-versus-unmerged benchmarks.

Before any production rollout, I would add strict JSON parsing, per-tool argument-schema validation, semantic checks for order IDs and refund intent, idempotency protection, audit logs and a human approval path for refunds. Any invalid or uncertain call should be rejected or escalated rather than executed.

5. Where the best model still breaks: five observed failures

The notebook selected the **base model** as the best overall model by exact-match score and exported the following five incorrect held-out examples. All five outputs were valid JSON; four were argument-value mismatches and one selected the wrong tool.

1. **Customer:** "Please begin the return-and-refund process for order ORD011469: the product does not match the description."
   Expected: `initiate_refund` with `{"order_id":"ORD011469","reason":"the product does not match the description"}`
   Predicted: `initiate_refund` with `{"order_id":"ORD011469","reason":"product does not match the description"}`
   Mistake: Argument value mismatch.
2. **Customer:** "Where does order ORD010479 stand in the fulfilment process?"
   Expected: `check_order_status` with `{"order_id":"ORD010479"}`
   Predicted: `track_shipment` with `{"order_id":"ORD010479"}`
   Mistake: Wrong tool; order fulfilment status was confused with shipment tracking.
3. **Customer:** "I need this case handled by a person. I need help with a complicated order problem. Reference case CASE011957."
   Expected: `escalate_to_human` with `{"reason":"I need help with a complicated order problem"}`
   Predicted: `escalate_to_human` with `{"reason":"complicated order problem"}`
   Mistake: Argument value mismatch.
4. **Customer:** "Please begin the return-and-refund process for order ORD011459: the order was cancelled."
   Expected: `initiate_refund` with `{"order_id":"ORD011459","reason":"the order was cancelled"}`
   Predicted: `initiate_refund` with `{"order_id":"ORD011459","reason":"Order was cancelled"}`
   Mistake: Argument value mismatch.
5. **Customer:** "Connect this conversation with an operator. I need someone to review my case. Reference case CASE011988."
   Expected: `escalate_to_human` with `{"reason":"I need someone to review my case"}`
   Predicted: `escalate_to_human` with `{"reason":"Customer needs assistance with their case."}`
   Mistake: Argument value mismatch.

The main pattern is argument fidelity rather than JSON formatting. Some reason-field differences are close paraphrases and are penalized by the notebook's strict string-equality metric; in a real system, a reason field may need semantic validation rather than exact wording.

The order-status versus shipment-tracking error is a genuine routing mistake and should be addressed with clearer intent boundaries and targeted hard-negative examples.

6. The agentic angle: one base, multiple specialist adapters

A router agent, refund agent and shipping agent can share one frozen base model while loading a small task-specific LoRA adapter for each role. This avoids maintaining three separately fine-tuned copies of the entire model. For one 4096 × 4096 weight matrix at rank 8:

- One LoRA adapter adds rank × (input dimension + output dimension) = 8 × (4096 + 4096) = 65,536 trainable parameters.
- Three adapters add 196,608 parameters in total.
- Three independent full copies of the same matrix contain 50,331,648 parameters.

For this single-matrix illustration, the shared-base design adds 256 times fewer parameters than three complete matrix copies. This is an illustration for one adapted matrix, not a whole-model storage measurement; the total depends on how many layers and modules receive adapters.

A real risk of switching adapters while requests are live is **cross-session adapter leakage**: one customer's request could accidentally be processed with another agent's adapter, policy or tool schema. The serving layer should pin an adapter version to each request/session, avoid unsafe global adapter switching, validate the selected adapter and schema before execution, and support rollback.

7. What surprised me

The most surprising result was that fine-tuning made function selection look almost perfect while making complete function calls less reliable. LoRA reached 100% tool accuracy, yet only 25% of its outputs passed the argument-schema check and achieved exact match. The baseline selected the correct tool less often (87.5%) but still produced more complete correct calls (41.5%). This shows why tool accuracy alone is not a sufficient evaluation metric.

The second surprise was that increasing LoRA rank did not improve exact match at all. The rank sweep changed parameter count and memory slightly, but every tested rank remained at 25% exact match. Finally, QLoRA's memory advantage was real in this run, but it was an efficiency improvement rather than a quality improvement.

8. Limitations and next steps

1. The current metrics come from synthetic template-generated data because only two Glaive examples passed the notebook's compatibility mapping. Real customer conversations and the original public dataset still need to be evaluated.
2. The 200-example test set and one random seed provide limited statistical confidence. Repeat the comparison with multiple seeds and a larger, real-world test set.
3. Exact-match scoring treats semantically similar free-text reasons as failures. Retain exact checks for tool names, required keys and IDs, but add a separate semantic evaluation for reason fields.

4. Merged and unmerged LoRA, QLoRA and DoRA artifacts were not compared. Inference throughput, time to first token, serving memory and end-to-end latency were also not measured.
5. No independent BFCL evaluation was run; no BFCL score is claimed.